Dynamic Bandwidth Allocation Project Report

Deep Learning-Based Dynamic Bandwidth Allocation for Intelligent Network Resource Management

Project and Team

Course	Computer Networks, B.Tech Year 2, Semester 3
Report date	28 October 2026
Project status	Simulator, dashboard, and 20-seed evaluation complete

Student Name	Registration No
Kavish Malhotra	25BDS0065
Sarthak Kamra	25BDS0077
Anuj Bohra	25BDS0090

Project overview

This project explores how a shared network link can be divided among interactive, web, video, and bulk-download traffic. A synthetic traffic generator creates changing demand; forecasting methods estimate the next time slot; allocation policies share the link; and a queue model records traffic served, waiting, and dropped. A local web dashboard makes the simulation and its results visible. The project does not read or control a real network connection.

Objective

The objective is to compare whether predicting near-future demand helps use a fixed link capacity better than simpler allocation methods, while also observing the trade-offs in fairness, queues, and estimated delay.

How the project works

Stage	What happens
1. Generate demand	The simulator creates Mbps demand for four traffic classes. Cycles, random variation, and bursts make the trace change over time; the selected scenario changes these patterns.
2. Forecast the next slot	Each predictor uses the previous 8 slots for all 4 classes (32 input values) to estimate the next demand for each class.
3. Allocate capacity	Six policies produce candidate allocations. Weighted water-filling uses class priorities and predicted demand, and keeps the total allocation within link capacity.
4. Simulate queues	Traffic enters a finite buffer for each class. The allocated service drains traffic; leftover traffic waits, and traffic that exceeds the buffer is dropped.
5. Compare results	The simulator records throughput, unmet demand, utilization, fairness, backlog, estimated delay, and dropped traffic for each policy.

The standard configuration uses a 100 Mbps link, 50 ms slots, and a 5 Mb buffer per class. The neural model is trained from scratch on generated standard-pattern traffic using NumPy.

Models, Queues, and Dashboard

The dashboard steps through one generated run and also displays precomputed multi-seed results.

Forecasts and allocation policies

Policy	Role
Equal Share	Divides capacity equally, without forecasting demand.
Max-Min Fairness	Shares capacity fairly using the latest observed demand, without priority weights.
Last-Observed + WF	Uses the latest slot as the demand estimate, then applies weighted water-filling.
Moving Average + WF	Uses the mean of the previous 8 slots, then applies weighted water-filling.
Neural Dynamic + WF	Uses a 32-48-24-4 feed-forward neural network, then applies weighted water-filling.
Oracle Reference	Uses exact next-slot demand. This ideal reference is not available to a real predictor.

Queues and measurements

Measure	Meaning
Throughput	Traffic successfully served, shown as a rate in Mbps.
Unmet demand	Requested traffic not served in the slot; it may remain queued or be dropped.
Utilization and fairness	Utilization is the share of link capacity used. Jain fairness is closer to 1 when service is more evenly shared.
Backlog and delay	Backlog is traffic waiting in queues. Delay is a simplified backlog-drain estimate, not measured packet latency.
Dropped traffic	Traffic rejected when a class buffer is full, expressed as Mbps rather than a packet count.

Dashboard operation

The parameter panel sets the scenario, link capacity, random seed, buffer size, slot duration, run length, and class priority weights. **Apply & Re-initialize** starts a new run with those values. The run length defaults to 600 slots; the first 8 slots provide predictor history, leaving 592 evaluated slots. At 50 ms per slot, that is 30 seconds of simulated time.

For a one-slot demonstration, press **Step 1: Predict** to inspect forecasts and planned allocations, then press **Step 2: Reveal Actual** to show generated demand and update the queues. **Step & Reveal** combines both actions; **Play**/**Pause** runs automatically; **+50 Slots** advances in a batch; and **Reset** starts over with the current settings. The chart shows recent class demand, link capacity, and the latest neural allocation. The tables below show run-level policy results, per-class results, and 20-seed benchmarks with 95% confidence intervals.

To launch it, run `python src/dashboard.py --port 5000` in the project folder and open `http://127.0.0.1:5000`. The playback interval controls the dashboard animation speed; it is different from simulated slot duration.

Evaluation and Findings

The evaluation uses 20 seeds (101-120) in four scenarios: standard mixed traffic, heavy bursts, capacity constrained traffic, and distribution shift. Each seed generates 600 slots; the first 8 provide history, leaving 592 evaluated slots per seed (11,840 evaluated slot observations per scenario). Values below are means across seed-level averages ± 95% t-confidence-interval half-widths.

Standard scenario results, 100 Mbps link

Policy	Throughput (Mbps)	Unmet (Mbps)	Jain fairness	Estimated delay (ms)
Equal Share	70.16 ± 0.89	7.64 ± 0.43	0.847 ± 0.005	38.7 ± 1.8
Max-Min Fairness	73.84 ± 0.95	7.25 ± 0.26	0.813 ± 0.005	125.5 ± 12.1
Last-Observed + WF	73.84 ± 0.95	7.28 ± 0.27	0.811 ± 0.005	114.2 ± 10.7
Moving Average + WF	73.82 ± 1.07	9.08 ± 0.23	0.825 ± 0.005	91.1 ± 2.6
Neural Dynamic + WF	73.67 ± 0.98	6.62 ± 0.42	0.823 ± 0.005	112.1 ± 27.1
Oracle Reference	73.91 ± 0.95	1.50 ± 0.28	0.811 ± 0.005	118.7 ± 15.4

What the results suggest

In the standard scenario, Neural Dynamic had the lowest mean unmet demand among the practical policies (6.62 Mbps), while throughput stayed close to the simpler baselines. In heavy bursts, neural unmet demand (38.23 Mbps) was almost the same as Max-Min (38.25 Mbps), while Moving Average had the highest throughput (93.42 Mbps). Under capacity constraints, neural unmet demand was lowest among practical policies (25.22 Mbps), but Moving Average had the highest throughput (74.81 Mbps). Under distribution shift, Neural Dynamic throughput (85.76 Mbps) was below Last-Observed (87.99 Mbps) and Moving Average (88.79 Mbps). No policy is best on every measure and scenario.

Limitations and conclusion

Traffic is synthetic, queues are simplified drop-tail buffers, and delay is estimated rather than measured from packets. The neural model is trained on standard-pattern traces and is less reliable when traffic shifts. The dashboard is not a live traffic monitor or a bandwidth controller. These results are descriptive comparisons; paired significance tests were not performed. The project demonstrates how forecasting and allocation policies can be compared in a controlled simulation, and future work could add authorized real-trace replay or a read-only live monitor.

Reproducibility

Dashboard	python src/dashboard.py --port 5000
20-seed evaluation	python src/evaluate.py --seeds 20 --slots 600 --output results
Saved outputs	Per-scenario benchmark CSV files and results/benchmark_summary.json